TraceAI — Enterprise GenAI Agent Platform

Complete Project Guide & Interview Preparation Document

TABLE OF CONTENTS

1. Project Overview
2. Problem Statement & Why This Project Exists
3. Tech Stack — What & Why
4. System Architecture
5. Project Execution Guide
6. Complete Feature Walkthrough
7. Key Engineering Decisions
8. Interview Q&A — 25 Questions
9. Resume-Ready Description

---

1. PROJECT OVERVIEW

Project Name: TraceAI — Enterprise GenAI Agent Platform

One-Line Description: A full-stack AI agent operations platform that orchestrates, observes, evaluates, and debugs autonomous AI agents with production-grade reliability.

What It Is: TraceAI is not a simple chatbot. It is an **AI Control Plane** — a unified workspace where you can:

- **Register** AI agents with different capabilities
- **Execute** natural language goals through an intelligent planning pipeline
- **Trace** every step of execution (planning, tool usage, synthesis)
- **Evaluate** agent quality with benchmark test suites
- **Debug** failures with structured error diagnostics
- **Monitor** system health in real-time

Type: Full-Stack Web Application (Monorepo: Python Backend + React Frontend)

Built By: Joylan Dsouza **GitHub:** https://github.com/Joylan9/genai-agent-sprint

---

2. PROBLEM STATEMENT

The Problem

Building AI agents is the easy part. **Operating them in production** is where every team struggles:

- When an agent fails, you see "*Error: something went wrong*" — no context, no trace, no fix.
- There's no way to compare agent performance over time.
- Debugging requires reading raw logs instead of visual timelines.
- Every agent framework gives you a runtime, but **zero operational tooling**.

The Solution

TraceAI provides the **missing operational layer** for AI agents:

Problem	TraceAI Solution
"What happened during this agent run?"	**Execution Timeline** — visual proportional bars showing planner → tools → synthesis
"Why did this run fail?"	**Error Panel** — What/Why/Fix/Retry with smart error categorization
"Is my agent getting better?"	**Evaluation Lab** — benchmark test suites with pass/fail metrics
"Is everything healthy?"	**System Status** — real-time health monitoring of all dependencies
"How do I navigate quickly?"	**Command Palette** — Ctrl+K fuzzy search + keyboard shortcuts

Who It's Built For

- AI Infrastructure Engineers
- GenAI Platform Teams
- Agent Systems Engineers
- Anyone building AI agents that need to work beyond a demo

---

3. TECH STACK — WHAT & WHY

3.1 Backend

Technology	Version	Why This Choice
Python	3.11+	Industry standard for AI/ML. Async support via `asyncio`. Massive ecosystem for LLM tooling.
FastAPI	0.128	Async-native REST framework. Auto-generates Swagger docs. Built-in Pydantic validation catches bad input before it hits your logic. 10x faster than Flask for async workloads.
Ollama	0.6.1	Local LLM inference — zero API costs, no rate limits, no vendor lock-in. Runs llama3 on your machine. No OpenAI dependency.
Celery	5.6	Distributed task queue for background agent execution. Prevents API timeouts on long-running goals. Workers can scale independently.
Gunicorn + Uvicorn	Latest	Production ASGI serving. Gunicorn manages worker processes, Uvicorn handles async requests. Standard production stack.
Pydantic	2.12	Data validation at the API boundary. Every request/response is type-checked. Prevents runtime crashes from bad data.

3.2 Frontend

Technology	Version	Why This Choice

React	19	Component-based UI with latest concurrent rendering features. Industry standard — every hiring team knows React.
TypeScript	5.x	Static typing catches bugs at compile time, not runtime. `npx tsc --noEmit` found 0 errors across the entire codebase — that's type safety in action.
Vite	7	Build tool with sub-second Hot Module Replacement (HMR). Native ESM — 10x faster than Create React App.
TanStack Query	v5	Server state management with automatic polling, caching, revalidation, and loading states. Replaces manual `useEffect` + `useState` patterns.
Tailwind CSS	v4	Utility-first CSS with dark mode support. No context-switching between CSS and JSX. Responsive design built-in.
React Router	v6	Client-side routing with nested layouts. The `AppShell` wraps all authenticated routes.
Lucide React	Latest	Consistent icon library (200+ icons). Tree-shakeable — only ships icons you actually use.

3.3 Infrastructure

Technology	Version	Why This Choice
MongoDB	7	Document store — perfect for flexible agent traces that vary in structure. No rigid schema migration needed.
Redis	7	Message broker for Celery + caching layer. In-memory speed for frequently accessed data.
Docker Compose	Latest	One-command infrastructure setup. Health checks ensure services start in the right order. Reproducible across any machine.
Prometheus	Latest	Metrics collection for LLM latency, request throughput, and error rates. Industry-standard observability.

3.4 Security & Quality

Technology	Purpose
JWT (Custom)	Stateless authentication — no session storage needed. Custom implementation using HMAC-SHA256.
bcrypt-style hashing	Password hashing with random salt — even if DB is compromised, passwords are safe.
OTP via SMTP	Email-based verification for password resets. 6-digit code with 10-minute expiry and rate limiting.
Flake8	Python linting — enforces PEP 8 code style.
Bandit	Python security scanner — catches hardcoded secrets, SQL injection patterns, etc.

Pytest	10 test files covering agent execution, guardrails, circuit breaker, routing, and E2E smoke tests.

---

4. SYSTEM ARCHITECTURE

How The System Works (Data Flow)

```
User enters goal: "What is quantum computing?"
        │
        ▼
┌──────────────────────────────────────────────┐
│  FRONTEND (React + TypeScript)               │
│  Playground Page → agentClient.runAgent(goal)  │
│  POST /api/agents/{id}/run                   │
└──────────────────────┬───────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────┐
│  API LAYER (FastAPI)                         │
│  ├── JWT Authentication Middleware           │
│  ├── Guardrails & Policy Engine (input check)  │
│  └── Dispatches to Celery Worker             │
└──────────────────────┬───────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────┐
│  AGENT RUNTIME (Celery Worker)               │
│                                              │
│  Step 1: PLANNER                             │
│  ├── Sends goal to Ollama LLM                │
│  ├── LLM decomposes into subtasks            │
│  └── Output: structured plan                 │
│                                              │
│  Step 2: ROUTER                              │
│  ├── Classifies each subtask                 │
│  └── Routes to appropriate tool              │
│                                              │
│  Step 3: TOOL EXECUTION                      │
│  ├── Web Search (DuckDuckGo API)             │
│  ├── RAG Retriever (Vector Store + MongoDB)  │
│  └── Calculator (math expressions)           │
│                                              │
│  Step 4: SYNTHESIZER                         │
│  ├── Collects all tool outputs               │
│  ├── Sends to Ollama LLM for final answer    │
│  └── Output: coherent response               │
│                                              │
│  Every step is TRACED and stored in MongoDB   │
└──────────────────────┬───────────────────────┘
```

```
                    │
                    ▼
┌──────────────────────────────────────────────┐
│  STORAGE                                      │
│  ├── MongoDB: traces, agent configs, user data  │
│  ├── Redis: task queue, response cache          │
│  └── Prometheus: latency & throughput metrics   │
└──────────────────────────────────────────────┘
```

Key Architecture Patterns

Pattern	Where	Why
Circuit Breaker	Ollama, Web Search, MongoDB calls	Prevents cascade failures — after N failures, breaker trips and returns fallback instantly
Repository Pattern	`app/memory/`	Abstracts database operations. Can swap MongoDB for PostgreSQL without changing business logic
Dependency Injection	FastAPI `Depends()`	`get_current_user` injected into protected routes. Clean, testable, no global state
Event-Driven Processing	Celery + Redis	Long-running agent jobs don't block the API. Workers scale independently
Policy Engine	`app/security/`	Validates plans and outputs against configurable rules before execution

---

5. PROJECT EXECUTION GUIDE

Prerequisites To Install

Tool	Install Command	Verify
Python 3.11+	python.org/downloads	`python --version`
Node.js 18+	nodejs.org	`node --version`
Docker Desktop	docker.com/get-started	`docker compose version`
Ollama	ollama.com/download	`ollama --version`
Git	git-scm.com	`git --version`

Step-by-Step Execution

STEP 1: Clone & Configure

```
git clone https://github.com/Joylan9/genai-agent-sprint.git
cd genai-agent-sprint
```

```
copy .env.example .env
```

Edit `.env` — set your MongoDB URI, Redis URL, and optionally SMTP credentials.

STEP 2: Start Infrastructure (MongoDB + Redis)

```
docker compose up -d mongo redis
docker compose ps        # Verify: both should show "healthy"
```

STEP 3: Start Ollama (Local LLM)

```
ollama serve             # Runs on port 11434
ollama pull llama3:8b-instruct-q4_K_M    # ~4.7 GB download, one-time
```

STEP 4: Start Backend (Terminal 1)

```
# Working directory: genai-agent-sprint/
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.api_app:app --reload --host 0.0.0.0 --port 8000
```

✅ Backend live at: http://localhost:8000 ✅ API Docs at: http://localhost:8000/docs

STEP 5: Start Celery Worker (Terminal 2)

```
# Working directory: genai-agent-sprint/
.venv\Scripts\activate
celery -A app.infra.celery_app worker --loglevel=info --concurrency=1
```

STEP 6: Start Frontend (Terminal 3)

```
# Working directory: genai-agent-sprint/frontend/
cd frontend
npm install
npm run dev
```

✅ Frontend live at: http://localhost:5173

STEP 7: Open & Use

Open http://localhost:5173 → Register → Login → You're in!

---

6. COMPLETE FEATURE WALKTHROUGH

6.1 Authentication System

Login Page (`/login`)

- **What to do:** Enter your email and password, click "Sign In"
- **Features:** JWT token-based auth, "Remember me" session, animated gradient background
- **Technical:** POST `/api/auth/login` → Returns access_token + refresh_token

Register Page (`/login` → "Create Account")

- **What to do:** Enter name, email, password → Click "Create Account"
- **Features:** Email validation, password strength meter, duplicate email check
- **Technical:** POST `/api/auth/register` → Creates user in MongoDB, returns JWT

Forgot Password (`/forgot-password`)

- **What to do:** Enter email → Receive OTP (6-digit code) → Enter code → Set new password
- **Features:** 3-step wizard with progress indicator, OTP auto-fill in dev mode, countdown timer for resend
- **Technical:** POST `/api/auth/request-otp` → POST `/api/auth/verify-otp` → POST `/api/auth/reset-password`

6.2 Dashboard (`/`)

What you see:

- **Stats cards** at the top: Total Agents, Total Runs, Success Rate, Avg Latency
- **Recent Agents** table: Shows registered agents with status
- **System Health** indicator in the top nav

What to click:

- Click any agent row → Goes to Agent Detail Page
- Click "View All" → Goes to Agent List
- Press `Ctrl+K` → Opens Command Palette

6.3 Agent Directory (`/agents`)

What you see:

- Table listing all registered agents with: Name, Model, Status, Created Date
- "Create Agent" button

What to do:

- Click **"Create Agent"** → Opens the agent creation form
- Fill in: Agent Name, Description, Model (e.g., llama3), Tools (web_search, rag, calculator)
- Click **"Save"** → Agent is registered and appears in the table
- Click any agent row → Opens **Agent Detail Page** showing full config

Empty State: When no agents exist, you see an animated Bot icon with "No agents registered yet" and a "Create Agent" call-to-action button.

6.4 Playground (`/execute`)

This is where you RUN agents.

What to do:

1. Select an agent from the dropdown
2. Type a goal in the text box, e.g.:
   - *"What is quantum computing and how does it differ from classical computing?"*
   - *"Search the web for latest news about artificial intelligence"*
   - *"Calculate 15% of 2500 and explain compound interest"*
3. Click **"Run"** (or press `Ctrl+Enter`)
4. Watch the response stream in

What you see:

- Loading state with animated spinner
- Final response rendered as formatted text
- If error: Styled error card with red border, AlertTriangle icon, and smart suggestion (e.g., "Check if Ollama is running")

6.5 Run History (`/runs`)

What you see:

- Table of all past runs: Run ID, Agent, Status (completed/failed/running), Duration, Timestamp
- Color-coded status badges (green=completed, red=failed, blue=running)

What to click:

- Click any run row → Opens **Run Details Page**
- Use pagination to browse older runs

Empty State: Animated Activity icon with "No runs found yet" and "Go to Playground" CTA button.

6.6 Run Details (`/runs/:id`) ⭐ MOST IMPRESSIVE PAGE

What you see:

1. **Header** with Run ID badge, duration, and "Share Run" copy-link button
2. **Error Panel** (only on failed runs):
   - Red gradient header "Execution Failed"
   - **Why** section explaining the error in plain English
   - **Goal Context** showing what the agent was trying to do
   - **Suggested Fixes** with lightbulb icons (e.g., "Check Ollama service", "Verify API key")
   - **Retry** button → Takes you to Playground with goal pre-filled
   - Collapsible raw error details
3. **Execution Timeline**:
   - Animated proportional bars: Planner (blue) → Tools (amber) → Synthesis (purple)
   - Hover over any segment → Tooltip shows exact duration and percentage
   - Total time displayed in top-right
4. **Trace Details** tab:

    - Full JSON trace of every step
    - Syntax-highlighted with line numbers
5. **Latency Breakdown**:
    - Metrics for planner_time, tool_wall_time, synthesis_time

What to click:

- **"Share Run"** → Copies the full URL to clipboard (shows ✓ Copied!)
- **"Retry"** on error panel → Navigates to Playground with the original goal
- Hover timeline segments → See timing details
- Click tabs → Switch between Trace, Logs, Artifacts views

6.7 Evaluation Lab (`/eval`)

What you see:

- Evaluation metrics summary: Total Tests, Pass Rate, Avg Score
- Test results table with individual test cases

What to do:

- Click **"Run Test Suite"** → Executes benchmark tests against the agent
- View pass/fail results per test case
- Compare scores over time

6.8 System Status (`/status`)

What you see:

- Dependency health matrix: Ollama, MongoDB, Redis, Celery — each with green/red status
- Response times per service
- Version information

6.9 Top Navigation Bar

Features:

- **Search bar** → Type and press Enter to search agents, or click to open Command Palette
- **Environment badge** → Shows "DEV" / "LOCAL" / "PRODUCTION"
- **System Health indicator** → Green pulsing dot when all systems OK, red when degraded
- **Activity button** → Shows count of active runs, click to open Activity Drawer
- **Toast notifications** → Pop up at bottom-right when runs complete or fail (auto-dismiss after 4s)
- **Dark mode toggle** → Moon/Sun icon

6.10 Command Palette (`Ctrl+K`) ⭐ POWER USER FEATURE

How to open: Press `Ctrl+K` anywhere in the app

What you see:

- Search box with fuzzy filtering
- **Navigations:** Dashboard, Agent Directory, Run History, Playground, Evaluation Lab, System Status

- **Actions:** Create Agent, Run Agent, Run Test Suite, Settings

How to use:

- Type to filter (e.g., "play" finds "Playground")
- Use `↑↓` arrow keys to navigate
- Press `Enter` to select
- Press `Esc` to close

Global Shortcuts (works anywhere, when not in an input field):

Key	Action
`R`	Go to Playground (Run agent)
`A`	Go to Agent Directory
`E`	Go to Evaluation Lab
`D`	Go to Dashboard

---

7. KEY ENGINEERING DECISIONS

7.1 Why Local LLM (Ollama) Instead of OpenAI API?

Decision: Use Ollama with local model inference instead of cloud APIs.

Reasoning:

- Zero API costs during development (OpenAI charges per token)
- No rate limits — can run unlimited tests
- No vendor lock-in — can switch models freely
- Full control over model quantization (Q4_K_M balances speed and quality)
- Works offline — no internet dependency

Trade-off: Requires decent hardware (8GB+ RAM). Mitigated by supporting CPU fallback.

Interview Answer: "*I chose local LLM inference because it gives us full control over the model lifecycle without external dependencies. This is the same approach companies like Meta and Apple use for on-device AI.*"

7.2 Why Circuit Breaker Pattern?

Decision: Wrap every external service call in a circuit breaker.

Reasoning:

- If Ollama goes down, without a breaker every request hangs for 30 seconds then fails
- Circuit breaker detects N consecutive failures, then **immediately** returns fallback
- Prevents one failing service from taking down the entire platform
- Self-heals: periodically retries to check if service is back

Interview Answer: "*I implemented the circuit breaker pattern from distributed systems design. It's the same pattern Netflix uses in their microservices to prevent cascade failures.*"

7.3 Why Custom JWT Instead of a Library?

Decision: Implement JWT from scratch using HMAC-SHA256.

Reasoning:

- Zero external dependencies for auth
- Full understanding of the token lifecycle
- Custom claims structure (access vs refresh tokens)
- Demonstrates deep understanding of security primitives

Interview Answer: "*I implemented JWT manually to demonstrate that I understand the cryptographic primitives — not just calling a library function. The token uses HMAC-SHA256 for signing, includes expiration checks, and separates access tokens (60 min) from refresh tokens (7 days).*"

7.4 Why Celery for Background Tasks?

Decision: Use Celery + Redis instead of running agent jobs synchronously.

Reasoning:

- Agent runs can take 10-60 seconds — would timeout REST APIs
- Celery allows async processing with status tracking
- Workers scale independently from the API server
- Redis provides both message brokering and result caching

Interview Answer: "*Long-running AI agent executions would block the API and cause timeouts. I used Celery with Redis as a message broker to decouple execution from request handling. This is the same pattern used in production ML pipelines at companies like Spotify and Instagram.*"

---

8. INTERVIEW Q&A — 25 QUESTIONS

Architecture Questions

Q1: Explain your project's architecture in 60 seconds.

> "*TraceAI is a full-stack AI agent platform with three layers. The React frontend communicates with a FastAPI backend through REST APIs. The backend has an agent runtime that uses a Plan-Execute-Synthesize pipeline — the planner uses an LLM to decompose goals, the router dispatches to tools like web search and RAG, and the synthesizer generates the final answer. Long-running tasks go through Celery workers backed by Redis. Everything is traced and stored in MongoDB for observability.*"

Q2: Why did you choose FastAPI over Flask or Django?

> "*FastAPI is async-native, which is critical for AI workloads that involve waiting for LLM responses. It auto-generates OpenAPI docs, has built-in Pydantic validation, and its dependency injection system makes testing clean. Django would be overkill for an API-only backend, and Flask lacks async support out of the box.*"

Q3: How does your authentication work?

> "*I implemented JWT-based stateless authentication. On login, the server creates two tokens — an access token (60 min expiry) and a refresh token (7 days). The access token is sent in the Authorization header on every request. A*

> *FastAPI dependency called `get_current_user` extracts and validates the JWT. For password resets, I use OTP via email with SHA-256 hashing, rate limiting (max 5 attempts), and 10-minute expiry."*

Q4: What happens when a user submits a goal?

> *"The frontend sends a POST request to `/api/agents/{id}/run` with the goal text. The API authenticates via JWT, validates input through guardrails, then dispatches the job to a Celery worker. The worker runs the Plan-Execute-Synthesize pipeline: (1) Planner sends the goal to Ollama LLM to decompose into subtasks, (2) Router classifies and dispatches each subtask to the appropriate tool, (3) Synthesizer collects all tool outputs and generates a coherent final answer. Every step is traced and stored in MongoDB."*

Q5: How do you handle errors in the agent pipeline?

> *"At the infrastructure level, I use the circuit breaker pattern — after N consecutive failures to a service, the breaker trips and returns a fallback immediately. At the UX level, I built an ErrorPanel component that categorizes errors using pattern matching — it maps error strings to human-readable categories like 'LLM Service Timeout' or 'Authentication Failed', suggests specific fixes, and provides a one-click retry with the original goal pre-filled."*

Frontend Questions

Q6: Why React over Vue or Angular?

> *"React's ecosystem is the largest, and its functional component pattern with hooks makes state management clean. I specifically chose React 19 for concurrent rendering features, paired with TypeScript for compile-time type safety. The component tree is straightforward: AppShell wraps all authenticated routes with a Sidebar and TopNav."*

Q7: How do you manage server state?

> *"I use TanStack Query (React Query). It handles API data fetching with automatic caching, background revalidation, and loading/error states. For example, the run history page polls every 8 seconds to detect new runs. This replaces the error-prone pattern of manual `useEffect` + `useState` + `isLoading` tracking."*

Q8: Explain your Command Palette implementation.

> *"It's a modal component triggered by Ctrl+K that lists 10 commands across Navigation and Actions categories. It has fuzzy search — typing 'play' matches 'Playground'. I implemented full keyboard navigation: up/down arrows to select, Enter to execute, Esc to close. I also added global shortcuts (R for run, A for agents, E for evals) that only fire when the user isn't in an input field."*

Q9: How do you handle empty states?

> *"I created a reusable EmptyState component with animated floating icons, accent color variants, and optional CTA buttons. It's deployed on 5 pages — Agent List, Runs, Evals, Dashboard, and more. This turns 'no data' from a dead end into a guided action."*

Q10: How does the Execution Timeline work?

> *"The RunTimeline component reads latency data from the trace — planner_time, tool_wall_time, synthesis_time — and renders proportional animated bars. Each segment width equals its percentage of total time. Tool segments are further split by individual tool observations. Hover shows precise timing in a tooltip. The animation uses staggered entrance (150ms delay per segment)."*

Backend Questions

Q11: What is the circuit breaker pattern and how did you implement it?

> *"The circuit breaker is a resilience pattern with three states: Closed (normal), Open (failing), Half-Open (testing). When a service fails N times, the breaker opens — all subsequent calls return immediately with a fallback instead of waiting 30 seconds to timeout. After a cooldown period, it enters half-open state and lets one request through. If it succeeds, the breaker closes. I applied this to Ollama, web search, and MongoDB calls."*

Q12: How does your OTP verification work?

> *"When a user requests a password reset: (1) Generate a 6-digit random OTP, (2) Hash it with SHA-256, (3) Store the hash in MongoDB with a 10-minute expiry, (4) Send the plaintext OTP via SMTP. When they submit the OTP: verify the hash matches, check expiry, enforce max 5 attempts (rate limiting), then issue a 15-minute reset token. The reset token is a JWT that can only be used to set a new password."*

Q13: How do you handle database operations?

> *"I use Motor (async MongoDB driver) through a MongoDB singleton class. The class manages connection pooling and provides type-safe access to collections. Agent traces, user data, and OTP records each have their own collection. I use MongoDB because agent traces have variable structure — document storage handles this naturally without schema migrations."*

Q14: What is your testing strategy?

> *"I have 10 test files covering: agent execution pipeline, input/output guardrails, circuit breaker behavior, intent routing, policy engine, retriever, health checks, and metrics. Tests use pytest with asyncio support. I mock external services (Ollama, web search) to test in isolation. There's also an E2E smoke test that validates the full pipeline end-to-end."*

Q15: How does the guardrails system work?

> *"The guardrails module validates both input and output. Input guardrails check for prompt injection, length limits, and prohibited patterns. Output guardrails verify the response doesn't contain sensitive information or toxic content. The policy engine validates plans against configurable rules before execution. All of this runs before and after the agent pipeline."*

DevOps Questions

Q16: How does your Docker setup work?

> *"Docker Compose orchestrates 4 services: MongoDB, Redis, API server, and Celery worker. Each has health checks — MongoDB uses `mongosh --eval`, Redis uses `redis-cli ping`, and the API checks its `/ready` endpoint. Services have dependency ordering — the API won't start until MongoDB and Redis are healthy. I also set memory limits (512MB for Redis, 1GB for API) to prevent resource exhaustion."*

Q17: How do you handle secrets management?

> *"All secrets are in `.env` files, which are in `.gitignore`. The application reads them via `os.getenv()` with fallback defaults for development. In production, secrets would be injected via Docker secrets or a vault service. The `.env.example` file documents all required variables without exposing actual values."*

Q18: What observability do you have?

> *"Three pillars: (1) Prometheus metrics for request latency, LLM call duration, and throughput — exposed at `/metrics`. (2) Structured logging with JSON format for easy parsing. (3) Health endpoints at `/health` and `/ready` that check every dependency. The frontend's System Status page shows all of this in real-time."*

Design & UX Questions

Q19: How did you approach the error UX?

> *"I believe error UX is a feature, not an afterthought. Instead of showing raw error strings, I built an ErrorPanel that categorizes errors using pattern matching. 'timeout' maps to 'LLM Service Timeout', '401' maps to 'Authentication Failed'. Each category has specific fix suggestions. There's also a retry button that navigates to the Playground with the original goal pre-filled — reducing the steps from error to recovery."*

Q20: What is your design system?

> *"I use Tailwind CSS v4 with a consistent design language: slate color palette, rounded-xl corners, gradient accents (blue-to-indigo for primary actions, emerald-to-teal for success, red-to-rose for errors). Dark mode support is built into every component. Animations use ease-out timing for natural feel. The system includes reusable components: Button, Input, Badge, EmptyState, ErrorPanel, RunTimeline."*

Q21: How does the notification system work?

> *"The TopNav polls active runs every 8 seconds. It tracks previous run states using a `useRef` map. When it detects a state transition (running → completed or running → failed), it triggers a toast notification. Toasts stack at the bottom-right (max 3 visible), auto-dismiss after 4 seconds, and can be manually closed. Green for success, red for failure."*

Behavioral / Design-Thinking Questions

Q22: What was the most challenging part of this project?

> *"Designing the execution timeline visualization. The challenge was making it both technically accurate (proportional timing) and intuitively understandable. I solved it by using proportional bar widths with minimum thresholds (so tiny segments are still visible), staggered entrance animations, and hover tooltips for precise timing. The phase flow arrows (Planner → Tools → Synthesis) give context even without hovering."*

Q23: If you had to scale this to 1000 concurrent users, what would you change?

> *"Three things: (1) Add Redis-based session caching for JWT validation instead of hitting MongoDB on every request. (2) Scale Celery workers horizontally with auto-scaling based on queue depth. (3) Add a CDN for the frontend static assets. The architecture already supports this — MongoDB, Redis, and Celery are all horizontally scalable."*

Q24: What would you add next?

> *"Conversational agent mode with multi-turn context. Currently each run is independent. I'd add session-based memory using MongoDB, a context window manager, and conversation history rendering in the Playground. The architecture already has a `session_id` field in the Trace interface — it's designed for this extension."*

Q25: Why should we hire you based on this project?

> *"This project demonstrates three things most candidates don't show: architectural thinking (circuit breakers, dependency injection, event-driven processing), operational thinking (health checks, metrics, structured error handling), and user-centered platform design (command palette, execution timeline, smart error diagnostics). I don't just build features — I build systems that operators trust at 2 AM."*

---

9. RESUME-READY DESCRIPTION

Project Title

TraceAI — Enterprise GenAI Agent Platform

One-Line Summary

Built a full-stack AI agent operations platform (React + FastAPI + MongoDB) featuring autonomous goal execution via local LLM, execution timeline visualization, benchmark evaluation lab, and enterprise authentication with OTP email verification.

Bullet Points for Resume

- Designed and built an AI Control Plane with Plan-Execute-Synthesize agent pipeline using Ollama (local LLM), achieving end-to-end goal execution with full trace logging
- Implemented production-grade backend with FastAPI, Celery, MongoDB, and Redis — including circuit breakers, rate limiting, and Prometheus observability
- Built React 19 + TypeScript frontend with 8 pages, Command Palette (Ctrl+K), animated execution timeline, smart error diagnostics (What/Why/Fix/Retry), and dark mode
- Developed JWT authentication system with OTP email verification, refresh tokens, and role-based access control
- Created Evaluation Lab for benchmarking agent quality with pass/fail metrics and historical comparison
- Containerized with Docker Compose (4 services) with health checks, readiness probes, and dependency orchestration
- Maintained 0 TypeScript errors across 14+ components and 10 backend test files covering E2E, guardrails, routing, and circuit breaker

Technologies to List

Backend: Python, FastAPI, Ollama, Celery, MongoDB, Redis, Prometheus, JWT, Docker **Frontend:** React 19, TypeScript, Vite, TanStack Query, Tailwind CSS, React Router **Testing:** Pytest, Vitest, React Testing Library **DevOps:** Docker Compose, GitHub Actions, Flake8, Bandit

---

Document prepared: February 2026 Project Repository: https://github.com/Joylan9/genai-agent-sprint